

Тема 1

1) Namespace - инструмент за избягване на коллизия на имена
• namespace smith; , smith::f, using namespace, using ::

2) Enumeration - избрани тип, списък с имена на променливи, които се държат като константи и съответстват на дадено цяло число

• enum - явно преобразуване в int, unscored (глобално)

• enum class - няма явно преобразуване, scored

• размерът на enum/enum class е колкото дългоименен тип, в който могат да се запишат

3) Структури - последователност от полета, които се запазват в определен ред в паметта; всичко е public по default

• показват се във функции A, const A*, A*, A*

4) размер на обекта

$\text{sizeof}(\text{клас/структ}) = \sum \text{глен-данните, които са примитивни} + \text{size на вложени класове като членове} + \text{падинги}$

• всички глен-данни трябва да запълват на число, или да са глен на $\text{sizeof}(\text{глен-данни})$. Винаги е по-добре да се налага падинга спрямо 1-ви елемент

5) Обединение - памет, която е обща за глен-данните

```
union A {  
    int x;  
    char y;  
}
```

ако $x = 65$,
то $y = 'A'$

// от ascii кода

x	x	x	x
5			

6) Endianness, // байт е цяло число от 0 до 255 → 256 бита

• Big endian - старши байт е в началото abc → abc

• Little endian - abc → cba

int u = 75

00 00 00 75 → 75 00 00 00

проверка: big/little endian

bool isLittleEndian {

union test {

uint32_t n=1;

unsigned char bytes [4];

myTest,

return myTest.bytes[0];

}

zag NTriangles

Работа с инстанциями: Point p;

• инициализация - $p.x=3$ или $\text{Point } p\{3,4\};$
 $p.y=4$

• доступ до элементов - $p.x$, $p \rightarrow x$ ако p е pointer
 $(*p).x$

• подаване във функ: $f(A\ a) \rightarrow$ по копие
 $f(A\&\ a) \rightarrow$ по реф
 $f(A^*\ a) \rightarrow$ по уе.
ако можем да избегнем копие;
ако не променяме обекта $\rightarrow$ const
отпред

• масиви - $A^*\ \text{arr} = \text{new } A[n];$ // delete[] arr;
 $A\ \text{arr}[5];$

• вложение - struct B {

A a1;

A a2;

}

• sizeof()

alignof() - sizeof(най-голямата тип-данна)

padding - отстояние за да може всяка данна да е на позиция на кейния size

Тема 2

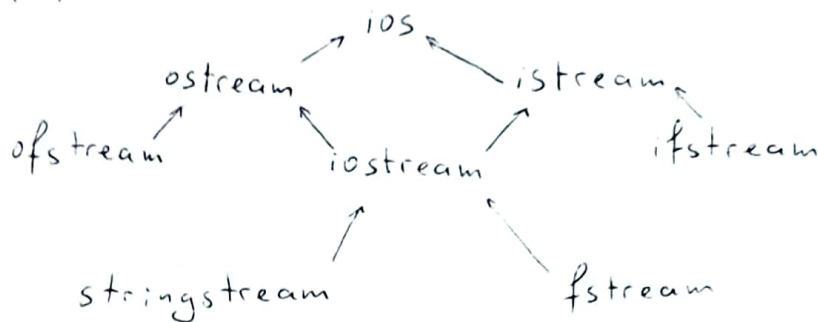
1) поток (stream) - последователност от байтове, данни влизащи в и излизащи от програмата

- потокът дефинира интерфейс с операции върху него, които не зависят от самите IO устройства

- поредност на работата

1. създава поток
2. свързва се с IO устройство
3. извършва операции върху потока
4. премахва връзка с потока
5. освобождава потока

- иерархия



- потоци за работа с конзолата (стандартен)

- вход (cin) - тип istream

- изход (cout) - тип ostream

- интерфейс на потоци

1) ostream

• формиран `os << обект>;` // връща os

• неформатиран `<< obj << endl`

2) istream

`>> write(char*, size_t bytes)` - за двоицен

• формиран `is >> обект>;` // връща is

- прескача бодещи табулации

- четене до space, \t, \n

• неформатиран

- `get()` - връща символ на който сме и мест

- `getline()` - за двоицен

- `peek()` - get без четене

- `getline(buffer, max_symbols, разделител)`

- `ignore(size_t / разделител /)` - прескача

// при ...

потоци за вход/изход от файл (текстови файлове)

1) изход $\rightarrow$ `ofstream ofs(fileName, режим на работа)`

- файлът като обект има буфер който периодично се синхронизира със съдържанието на файл. Можем експлицитно да го направим с `flush()`. При затваряне на файла с `close()` се извиква автоматично `flush()`.

2) вход $\rightarrow$ `ifstream ifs(-1, -1)`

позициониране

1) ostream

- `tellp()` $\rightarrow$ къде е put указателя
- `seekp(ind)` $\rightarrow$ местя put указателя на ind
- `seekp(offset, direction)` $\rightarrow$ местя put указателя с offset
относно от direction

directions:

- `ios::beg` - начало
- `ios::cur` - текущо
- `ios::end` - край

2) istream $\rightarrow$ `tellg()`, `seekg(ind)`, `seekg(offset, direction)`
аналогично, но с get указатели

Режим на работа: двоично число, "звонгале" с "1" двоично или

1) `ios::in` $\rightarrow$ четене

2) `ios::out` $\rightarrow$ писане

3) `ios::ate` $\rightarrow$ put ука. е накрая и може възникване на произволни места, писане

4) `ios::app` $\rightarrow$ put ука. е в края, писане

5) `ios::trunc` $\rightarrow$ изтрива съдържанието на файла

6) `ios::binary` $\rightarrow$ двоичен режим

7) `ios::noexist` $\rightarrow$ отваря само ако вече съществува

8) `ios::noexcept` $\rightarrow$ отваря само ако не съществува

флагове

1) `bad()` има загуба на инф. Операция не е изпълнена

2) `fail()` последната операция е невалидна

3) `good()` - всичко е минало успешно

4) `clear()` - изсти флаговете и отива на `good()`

5) `eof()` - достигат в края на файла

Тема 3

Работа с `fstream` - има 2 вида на `ifstream` и `ofstream`

- има само 1 указател
- след всяка операция трябва да `flush`-нем преди въвеждане
- `seekp(tellp)`
- `seekp = seekg`, `tellp = tellg`.

Двоичен файл - гуглен за четене от нас, летен за програмата

1) писане - `ofstream ofs("...", std::ios::out | std::ios::binary);`
`ofs.write((const char*)a, sizeof(a))`

използваме `char*` защото `sizeof(char) = 1 byte`, което е най-малката единица, която може да се чете/записва

2) четене - `ifstream ifs("...", std::ios::in | std::ios::binary);`
`ifs.read((char*)a, sizeof(a))`

• писане на масив

`ofs.write((const char*)ptr, sizeof(обекта от който) * size на масива)`

`// int arr = new int[N];`

`// ofs.write((const char*)arr, sizeof(int) * N)`

• четене на масив

1) намиране големината на файла - `fileSize`

2) `ifs.read((char*)arr, fileSize)`

• писане на обект използвайки динамична памет

`ofs.write((const char*)nameLen, sizeof(nameLen));`

`ofs.write((const char*)name, nameLen); // char* name`

• четене на динамичен обект `// char* name`

`ifs.read((char*)nameLen, sizeof(nameLen));`

`result.name = new char[nameLen + 1];`

`ifs.read((char*)result.name, nameLen);`

`result.name[nameLen] = '\0';`

Тема 4

- Указател `this` - указател достъпен в рамките на не-статичните член-функции на клас / struct. Соди към обекта, в който се викат член-функции
- Член-функции - функции в тялото на клас работят с член-функции на класа / struct. Извикват се от обект от класа
- компилаторът превръща всяка член-функция в обикновена функция с уникално име и един допълнителен параметър - константен указател към обекта - `this`

пример struct X {

void f() { ... } → void f(X* const this) { }

• константни член-функции

- не променят член-данните
- указва се чрез ключовата дума `const` след функцията .. `f() const`
- не могат да извикват не `const` функции

константна void f() const → void f(const X* const this)

не константна void g() → void g(X* const this)

// даме са `X::f` и `X::g(...)` ама да

• конструктор - член-функция, която се извиква при създаване на обекта

- има същото име като класа
- няма тип на връщане
- ако няма параметри се нарича default-ен
- ако не разпиеме статичен ще е `static default-ен`
- при създаване на масив от класа се викат деф. статични и размера на масива
- при наследяване ако не извикаме статичен на наследяващия се вина дефолтния

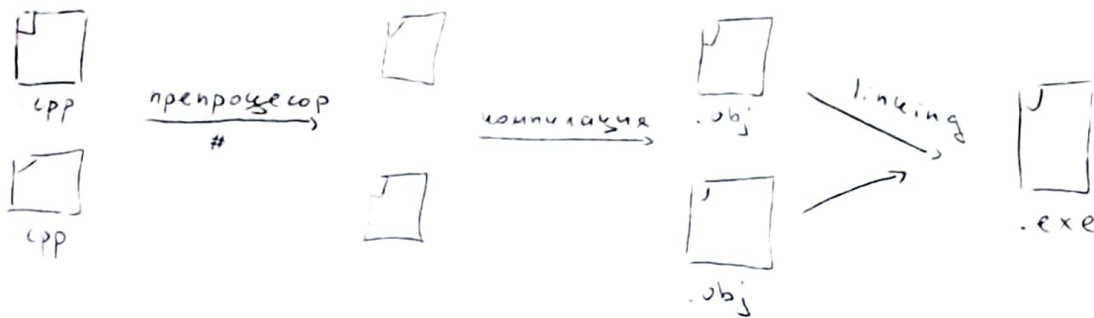
- деструктор - клас-функция, извиква се при изтриване на обекта
 - не се указва тип на връщане
 - има същият име като класа, но с ~ отпред
 - ако не разпознае ще се създаде default-ен
 - може да има само 1 дестр, без параметри
 - използва се за освобождаване на ресурси (динамични)
 - статичните данни си висят ~ в края на класа, динамичните не
- конвертиращи оператори
 - оператор с 1 параметър
 - explicit извади, че оператор НЕ е конвертиращ
- Модификатори за достъп
 - 1) public - имаме достъп отвсякъде
 - 2) private - имаме достъп от класа само
 - 3) protected - имаме достъп от класа и наследниците му
- Мутатори / селектори
 - 1) get - връща по копие, const, не променя само гледа
 - 2) set - променя, връща по реф
- абстракция - използваме нещо без да ни интересува как работи; скриване на нежунните детайли
- капсулация - ограничаване на достъпа
- mutable - позволява на член на обекта да предефинира константността си. Дори в const функ, данната ще може да е променена
 - използва се само в краен случай

Важно! Различие между const и mutable

• създаване на масиви $A \text{ arr}[3] \rightarrow A(1), A(2), A(3)$ - в края ~ $\sim A(1), \sim A(2), \sim A(3)$

за динамичен масив $\rightarrow A^* \text{ arr} = \text{new } A[3] \rightarrow \text{delete}[3] \text{ arr}$ не чиб. ~

Тема 5



- всеки `.cpp` файл се компилира отделно при промяна (разделна компилация)
- 1) препроцесора заменя всички `#include` и тн с текста в тях
- 2) ползва се `.obj` след компилация, което е машинен код
- 3) след `linking` се ползва извънния файл `(.exe)` ~~или linker~~ // linker
ако има проблеми се ползва linker грешки (link)
- за да се възползваме от разделната компилация - `.h` и `.cpp` и ако работим с класа само `include`-ваме `.h` файла
- компилационен процес (пореждност):
 - 1) препроцесор (текстобработка)
 - 2) токенизация (премахва интервали, табулации и тн)
 - 3) синтактичен анализ
 - 4) семантичен анализ (смыслов)
 - 5) оптимизации
 - 6) assembly code
 - 7) linking

`#include` - замества се с файла

`#define P 314` - замества `P` с числото 314

`#pragma once` - за да не се изпълнява 2 пъти едно нещо (винаги в `.h`)

• копиране на обекти с `CC` и `or =`

- копиращ стат - стат, който приема обект от класа и създава новия обект като копие

// `A(const A& other)`

- `operator =` - рунин / оператор, който приема обект от същия клас и променя данните на вече съществуващ обект от същия клас

// A2 operator = (const A2 other)

- ако не сме разписали се създават default-ни

- `сс` създава нов обект, `ор` = променя съществуващ

• RVO (return value optimization) - няма да се извикат `сс` или `ор` ако директно връщаме копие обект

```

A create() {
    return new A();
}

int main() {
    A obj = create();
}
    
```

// само `сс` и `~`

• shallow copy - при работа с външни ресурс (динамични данни и дефолтно генерирани `сс` и `ор`)

• композиция

```

class X {
    A a;
}
    
```

→ `a` зависи от `X` и се контролира от него

• UML - diagram



• агрегация

```

class X {
    A* a;
    A2 a2;
}
    
```

→ `a` и `a2` не зависят от `X` и няма да се унищожат при `~`

• UML - diagram



// композиция - , агрегация - 

заг клас Event с Time и Date готово

Тема 6

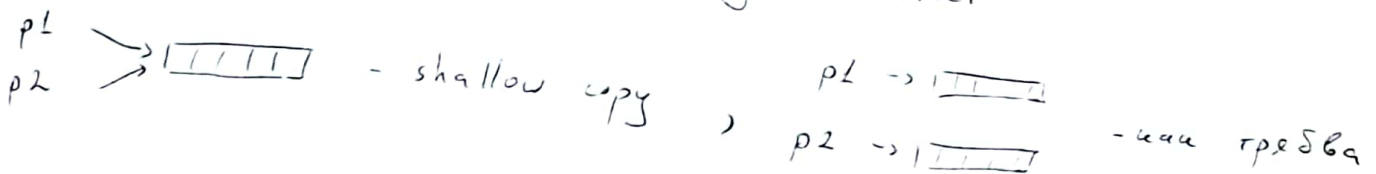
• Голяма сетворна

- default-ен стог
- cc
- op =
- ~

ако не ги разпознаем се генерират деф. (освен стог ако чинам гр или деф)

• Всички от тези 4 функции извиква същата ф-я рекурсивно нагоре

• проблем при работа с динамичната памет - ще копираме указателя, а не данните зад него (shallow copy). Така двата обекта ще споделят една памет



• помощни private функции: free(), copyFrom()

1 A() = default

2 ~A() {

 free();

3 A(const A& other) {

 copyFrom(other);

4 A& operator = (const A& other) {

 if (this != &other) {

 free();

 copyFrom(other);

 return *this;

Заг Конструктор с указател и оценен

Тема 7

предефиниране на оператори - дефиниране на поведение на оператор
в/у даден тип. ключова дума operator и символа, които приемат

• видове оператори:

- унарни (на 1 арг. - ++, !)
- бинарни (на 2 арг. - +, -, *, /)
- тернарен (? :)

• всички оператори приемат точно определен брой параметри освен операторът `()` // function call

• ако оператор е предефиниран във класа или в наследяващия клас параметърът `this` - обекта, който го вика

• Четири символа (+, -, *, &) са едновременно унарни и бинарни зависи от броя параметри

• характеристики на операторите

- приоритет - определя последователност, в която се извършват

- асоциативност - лява асоц. ($a+b+c \Rightarrow (a+b)+c$)

- дясна асоц. ($a=b+c \Rightarrow a=(b+c)$)

- позиция спрямо аргумента - префиксни, инфиксни, суфиксни

! при предефиниране на оператор не могат да се променят приоритета, асоциативността, броят и позицията спрямо аргументите му

• членове на клас

- ако се извършват 10 операции в/у класа се предеф. оп `>>` и `<<`

- ако се извиква проверка за равенство се деф. оп `==` и `!=`

- ако се извиква илюминация наредба се деф. оп `<` и заедно с него (и през него) всички останали оп за релации (`>`, `!=`, `>=`, `<=`)

• релации и логически и релационни (`<`, `==`, `!=`) - връщат bool

- аритметичните (+, -, ...) - съответно от същия тип, не реф

- присвояващи (`=`, `+=`) - реф към левия операнд

- може да се викат и като $operator+(x)$ или $op+(x)$ като се описва функцията
- предефиниране операторите като се променят някои оператори във класа $(+, -, =, \dots)$
- преустройството, които променят единичен операнд, привождане, индексация, `func call`, `member access` и т.н. вътре в класа $(=, +=, [], (), \rightarrow)$
- `op <<` и `>>` са викани външна функции, защото първият е левия аргумент, а вторият ни е десния
- приятелски класове и функции
 - ключова дума `friend`
 - добре е да са в `public` частта
 - не се наследяват
 - `>>` и `<<` са `friend`
- определение - външен клас / ф-я, който има достъп до `private` и `protected` членове на някой клас

Тема 8

Static

- локални променливи

```
void f() { static int x = 1; ... }
```

и сега на друго място в паметта - при глобалните променливи; инициализира се веднъж при първоп визити в

- стат-данни

```
class X { static int n; ... }
```

не се обвързва с конкретен обект

⇒ всички обекти от X (инстанции на класа) имат обзг и

- стат-функции

```
class X { static void f(), ... }
```

- няма достъп до стат-данните и стат-функции които не са static

Exception (Изключение) - сигнал, че е възникнал проблем

- използваме ключовата дума throw

Обработка на грешки

```
try { ... } catch (...) { ... } finally { ... }
```

Ако хванем грешка ще се извика дестр на всеки създадени обекти (stack unwinding). Ако не бяхме го хванали в try-catch блок нямаше да се извика дестр.

assert - зависи от кода, който ние сме написали

exception - зависи от външните данни

хубаво да сложим първо по-конкретните екземпляр и каква catch (std::exception ex) и catch (...) , които 3 тогава хващат всеки възможен (зафр. обект и т.)

хващане винаги по реф. catch (std::exception & ex)

• Иерархия на изключения

- exception базовия (std)
- bad-alloc - грешка при заделване на памет
- bad-cast - грешка при Casting
- logic-error
- runtime error

• Изключения в ctor

- ще се извикат дестр на все създадените обекти (за първата динамична динамична няма нито от try-catch нито за всички следващо, че не можем нито да има нито за горните) bad-alloc ще е най-вероятната грешка

• Изключения в destr НЕ правим

- Ако има 2 екземпляра се вика std::terminate() и програмата спира

```
Pr ~A() { throw 5; }  
void g() { throw 6; }  
int main {  
    A a;  
    try {  
        g();  
    }  
    catch (...) {}  
}
```

• Нива на exception safety

- no throw guarantee -> няма как да се върни екземпляр
- strong exception safety -> може да се върни грешка, но обекта няма да се промени
- basic exception safety -> може да се върни грешка, но обекта ще е използваем
- no exception safety -> нямаме гаранция за нищо

Тема 9

• A^* arr = new $A^*[n]$, $\rightarrow \underline{A^*} \mid A^* \mid A^* \mid A^* \mid A^*$

+ locality - последователно са в паметта

• A^{**} arr2 = new $A^*[n2]$



масив от указатели

- + resize не прави копия, а преназначва
- + може да има произни позиции
- + swap-ове се по свичи
- + трябва по-малко памет
- + няма кингда от дефолтен съг

ако променяме ел. от масива с vec е по-добре да модифицираме
който копират в vec вместо да създаваме нов масив и vec стария

Move семантика

- lvalue - притежават адрес в паметта (променлива, обект, функ, ...)
- rvalue - не са lvalue

• rvalue - литерали, ф-ии върнали по копие

• xvalue - обект към края на живота си (expiring value)
не можем да им приписваме стойности

lvalue ref е $\&$, rvalue ref е $\&\&$

rvalue ref могат да имат към временни обекти, не и lvalue.
lvalue ref и xvalue с функ. `std::move()` и
каква та коства на компилатора подаденото lvalue да се
третира като xvalue

lvalue ref - цел да можем да презеф една функ да
работи по различен начин в зависимост от това дали
обектът е lvalue или rvalue

Move stor

идеята е да отразим резултата на обект който няма да бъде използван след това но да го оставим в такова състояние, че изтриването му да не повлияе на нашите данни

- при примитивните типове е просто копиране
- при издателите - вземим нашите данни да имат или същата памет като тези които крадем, а уруги обект данните да ~~станат~~ имат пилот за да няма отегча и за да не изтрива отразените данни

Move op =

логиката за крадене на данни е същата като при MS, но тук първо изчисляване, ако няма вече данни

- Компиляторът ще създаде вон MS и move op = ако ~~не~~ не сме различали наши сс, сор op = или дестр и обектите могат да бъдат "пренесени"
- Ако един клас или move операция и използва копираци (комет и опитване да изберем)
- при примитивните данни местенето е просто копиране
- Ако спотим = default пред move операцията, то компилаторът не успее да ги създаде, ще ги обозначи с = deleted.

std::move(obj)

третира obj като хвалне, но само еднократно
след тази функ obj или ще бъде и се третира като такова

Тема 10

Наследяване (is a relationship) - дефиниране нов клас,

който има същото поведение и глен-данни като друг
и затова преползваме код и обхващаме нашия за
наследник на другия

- наследяват се глен-данните и функи на базовия клас, но
имаме достъп до някои (public, private) данни в базов

class B: A

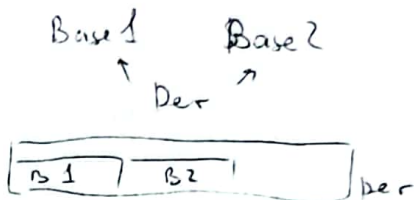
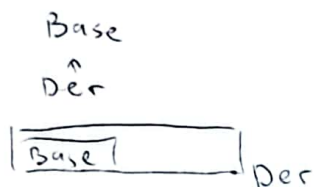


създават се нещата на A и
наследят на B

- Видове наследяване - private (подразбираше за класове),
protected,
public (подразбираше за struct)

видимостта става макс. във вида на наследяване, т.е. ако наследник protected класа, то всички public глен-данни се приемат до protected видимост в детен клас

• UML диаграма



Ако в Base и в Der има променлива с едно име, то
тази в Der shadow-ва тази в Base

Ако Base има глобален памет, той трябва да си е мекат, и
Der да не се занимава с нея и изтриването и т.н

Подаваме като параметър на функи - Der може да бъде подаван
като аргумент на функи, която очаква клас тип Base, но
ще се види само Base заиста на Der

проблем при масиви.

Base * е

12	13	0	0	0
----	----	---	---	---

Der * е

12	13	13	0	13	0	13	0
----	----	----	---	----	---	----	---

и ако използваме Der* компиляторът не знае, че трябва да прескочи данните на Der след Base, за да стигне до следващото Base и няма да работи коректно

- При конструиране трябва да се остане кой стъг на базовия клас да се извика, иначе се вика дефолтния му ако има такъв
- Деструкт. на наследника ще извика този на базовия
- Важно е поредността, в която сме наследили класовете, защото независимо как ги пермутираме в стъг на Der, ще се извикат както са наследени
- Ако иначе `cs` и `op =` в Der задължително трябва в тях експлицитно да извикаме този на базовия клас
`cs` и `more op =` е също - пак викаме експлицитно този на базовия (с `std::move(other)`)

~~параметри на функции (с и реф)~~

Тема 11

Полиморфизъм - едно име (интерфейс), различни имплементации

Статично свързване - изборът на функции (обект), който да се извика става по време на компилация

- зависи от типа на указателя / променливата през която се вика

Динамично свързване - изборът става по време на изпълнение на програмата (runtime polymorphism)

- изпълнява се чрез виртуални функции

// статично: 3^*b , $4^*a=b$; $b \rightarrow f() / 3()$, $a \rightarrow f() / 4()$ → следва се ртг вляво

// динамично: 1^*a ; 2^*b ; $a=2^*b$, тогава се гледат къде къде кой е обекта

- класове в които ще се прилага runtime

polymorphism трябва да са наследници на един и същ

- дефинират се виртуални методи, съответства на полиморфно действие и всеки от наследниците избира дали да го предефинира или не

- извикването става чрез указател към базовия клас и тогава през него ще извикват обектите задължително деструйт. на базовия трябва да е виртуален (ако има динамични данни е "чужди" и clone функция)

- ключова дума virtual през функции

Абстрактен клас - предизначен само за наследяване - не можем да правим обекти от него. Всеки клас, който го наследява и го развива своя имплементация на виртуални функции. също става абстрактен

virtual void f() = 0, ако има = 0 е виртуален и класа става абстрактен

Виртуални таблици - таблици с указатели към функции

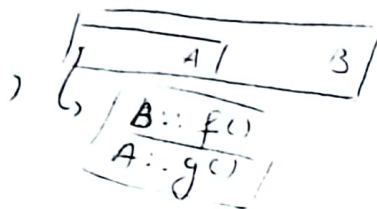
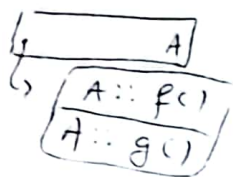
- те са за клас, а не за обект

- за вършат с pointer, за да знаем къде е крият

Пример:

```
struct A {  
    virtual void f();  
    virtual void g();  
};  
  
struct B : A {  
    void f() override;  
};  
  
struct C : A {  
    void f() override;  
    void g() override;  
};
```

Виртуални таблици (vTable)

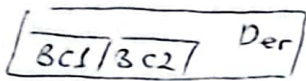


Override и final

- override - указва, че дадена функция прегришава функция от базовия клас. Ако в базовия клас няма такава функция, кодът няма да се компилира
- final - указва, че дадена функция не може да се override в нито един от наследниците в иерархията
 - ако е за клас - той не може да бъде наследен

Тема 12

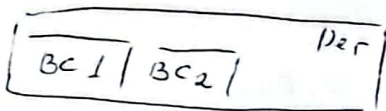
Множествено наследяване - можем да наследяваме повече от 1 клас



В конструктор както и да перипутираме базовите съб-и, ще се извикат както си наследени
 при op = обаче има значение как сме ги написали и затова трябва пак да ги пишем както сме ги наследили

Ако в два или повече класа имаме функции с едно и също име сме длъжни да реализираме имплементация в Der

Δ - какво отговаря да направим уе. за да имаме правилния обект



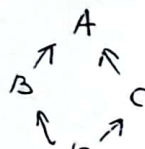
Der :: func	Δ(0)
BC1 :: func	Δ(0)
BC2 :: func	

→ Δ(select(BC1))

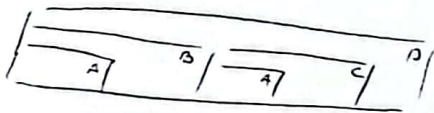
при множествено наследяване VT са наредени двойки → func/Δ

• Диамантен проблем

- когато наследяване едновременно с два или повече класа, които наследяват



проблем:



"правилно":



ка дъщеря A трябва да е обик

Решение: виртуално наследяване. Ако клас X наследява клас Y виртуално, то V който наследява X е отговорен за поддържане на Y

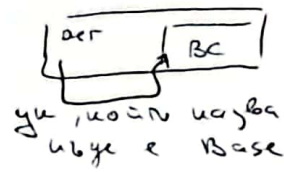
В нашия случай трябва



D(): B(), C(), A()

Ако няма A() - дефолтна

при виртуално наследяване се разменят местата в таблицата и виртуализацията е отзад, а не напред отпред



Колекции от обекти в полиморфна йерархия:

- хетерогенен контейнер - клас, който съдържа колекция от указатели към (абстрактен) клас. Ук. трябва да са от тип базовия клас на някоя полиморфна йерархия

- копиране - функ. clone, която връща копие на обекта, от който е извикана

- virtual Base * clone() const = 0, и наследниците override-ват

- копирането изглежда така: obj = other.obj -> clone();

- при наследниците: Base * clone() const override {
return new Der(*this);
}

- Триене - по време има виртуален дестр в базовия клас - ще се извикат дестр правилно на обектите в колекцията (защото са от тип базовия клас)

- virtual ~Base();

Тема 13

Шаблони - функ./класове/статист с общо предназначение
(параметричен полиморфизъм)

template <typename T>

T* агг ; T obj ;

- шаблоните могат да извикват необходими функ., за да ги използваме с наши клас, тоест клас, който ще се използва в шаблона (който се замества с T) за ина нужните предефинирани оператори и тн
- при извикване на шаблонна функ./клас, кода на шаблона се копира и се замества 'T' с типа, който сме ни
- // може да не е точно 'T', но е взето от Template

- Темплейтни класове - класове, в които някои елементи не са с ясен тип

пишем template <typename T> код деф. на класа и
можем да използваме T когато е като тип

- НЕ разделяме на .h и .cpp файл както нормално, защото .h файлът не може да сигнализира на .cpp файла с кой тип да генерира код.

1 вариант - пишем ги в общ файл с разл. разширение (пр. .h)

2 вариант - разделяме ги на .h и .cpp, но в края на .cpp файла пишем с кои типове ще работи и за всеки от тях ще се генерира по един .obj файл

// template <int> ; template <double> ; // = <int>

- близки шаблони се заместват по време на компилация
- Темплейтна специализация - функ./клас, който се върти различно за конкретен тип, а с останалите си се върти по единствен уруг начин

// template <T> вместо template <typename T>
 // void A<int>::f() { } void A<T>::f() { }

Примери за шаблонни класове:

- 1) vector<T> → темпл. клас за bool
- 2) std::sort() → разн. за разн. типове
- 3) queue, stack

• Умни указатели - обвивам клас около умн.; дестр. на умния умн. не изтрива данните на умн.; избягваме да пишем new и => delete

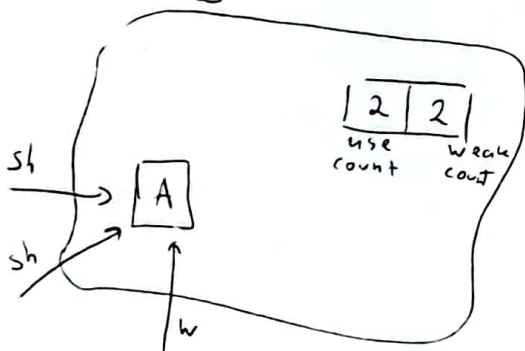
• първо е бил auto_ptr, но вече не се използва

• unique_ptr - един умн. за един обект, нямаме ss и op=
 // unique_ptr<A> ptr = make_unique(A(...));
 - дестр. на умн-ptr вина дестр. на обекта

• shared_ptr - последния умн. когато се триве изтрива и обекта
 // make_shared<A>(...)
 - нямаме ss и op=

• weak_ptr - non-owning ref, т.е. насочваме weak_ptr към обект, който е мекантиран от shared_ptr
 - не влияе на триенето на обекта
 - може да сочи към вече изтрят обект
 - може да се повиши в shared_ptr ако не е изтрят обект

Имплементация:



и двата ptr имат достъп до counter-и

use_count	2	1	2
weak_count			

- колко shared_ptr
 и колко weak_ptr
 имаме

0 - ако use_count = 0
 1 - ако use_count ≥ 1

Трим обекта - use_count = 0

Трим броя - weak_count = 0

Тема 14

Type casting - 4 вида

- static-cast - когато сме сигурни във вярния тип на състояние не правим runtime-checks

// static-cast <Base*> (ptr/obj);

- dynamic-cast - когато не сме сигурни в типа
 - прави runtime checks и е по-бавно
 - ако не успее ще върне nullptr (ако е ptr) или ще върне std::bad_cast ако е & (ref)
 - използва се за downcast-ване, но за да работи трябва базовия клас да има поне 1 виртуална функция

// Der* ptrD = dynamic-cast <Der*> (ptrBase);

- const-cast - можем да манипулираме константността на обекти
 - undefined behavior { - ако имаме константен обект / кон-данни, но искаме да променим стойността това става с този променен ключ
 - ако имаме const func, която искаме да променим

for кон-данни int num;

void f() const {

(const-cast <A*> (this)) -> num = 5; }

- ако искаме да модифицируем const data на ф-я, която приема const

- reinterpret-cast - преобразуване на указател (ptr) от произволен тип в указател (ptr) от произволен тип

strong cohesion - елементите да са стриктно обвързани в своя сигнатура и functionality. Без излишни кон-данни и функции в клас (които не са стриктно обвързани с него и не показват негово поведение).

loose coupling - low degree of dependency, всяка част/клас е максимално независима от другите

Solid principles

1. Single responsibility - 1 компонент има 1 отговорност
2. Open-closed principle - отворен за разширения, затворен за модификации
3. Liskov substitution - базовия клас трябва да е заменен от наследник
4. Interface segregation - разделяне на големите интерфейси на по-малки
за да не предоставяме ненужни методи при имплементация
Не трябва да зависи от коз, който не използваме
5. Dependency inversion - модулите от високо ниво не трябва да
зависят от модулите от ниско ниво - трябва
да зависят на абстракция

Design patterns - добри практики, решения на често срещани проблеми, 3

1. Creational patterns - осигурява начин за създаване на обекти
 - Factory
 - Singleton - инициализация само 1 обект от класа
- private static, delete с и op=, п-я getInstance()
2. Structural patterns - създаване на по-сложни класове чрез
наследяване/композиция
 - Flyweight pattern - събира повтарящи се обекти в паметта чрез
споделяне на общите им части, по-бързо, обектите
са immutable
3. Behavioral patterns - за комуникацията м/у обектите
 - Iterator - обхождане на колекция без да се интересуваме
каква е тя
 - Command - използваме клас, който да обвива request-ите
и да съдържа и енкапсулира всяка информация
- подаване request-а като параметър на функция

№1 class Command {

virtual void execute() = 0;

}

class Command1 : public Command {
void execute() override
{ ... }

}

и за вс команди, инициализация и да е тя ще инициализира с execute()